PRÁCTICA 1

Videoteca: de la ventana a los ficheros

Acceso a Datos (DAM) — UD1 Manejo de ficheros

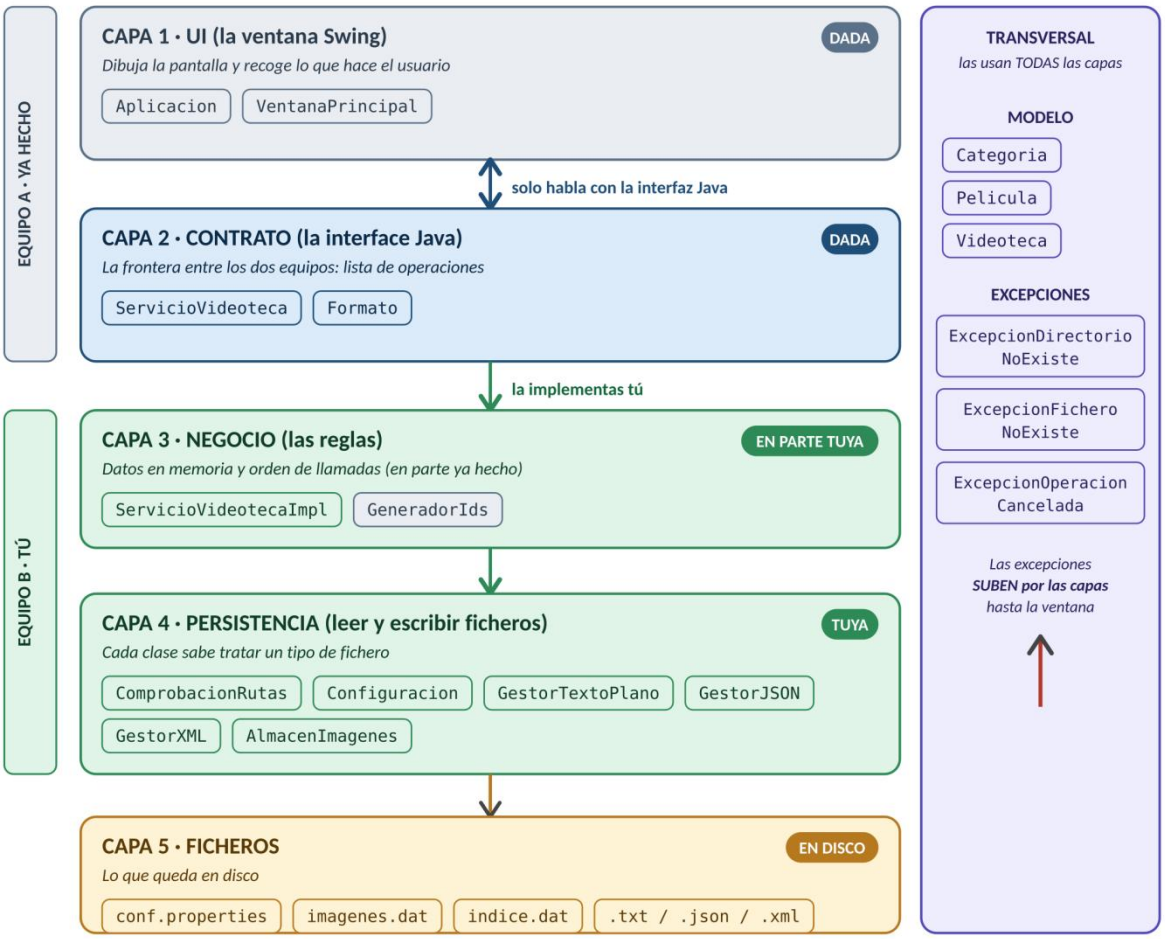
1 · LA IDEA DE LA PRÁCTICA

- Hay **dos equipos de desarrollo**. El **Equipo A** (ya hecho) te da la **ventana** y el **contrato de operaciones** (una `interfaz` Java).
- Tú eres el **Equipo B**: tu tarea es programar **lo que hay por debajo del contrato**.
- La UI **solo habla con el contrato**. No tienes que tocar esa parte.
- La videoteca **trae datos de ejemplo en memoria** para que puedas usarlos para realizar la **exportación inicial**.
- Orden de trabajo aconsejado: **exportar → importar → configuración e imágenes → crear datos**.

2 · EL MAPA: CAPAS Y EQUIPOS

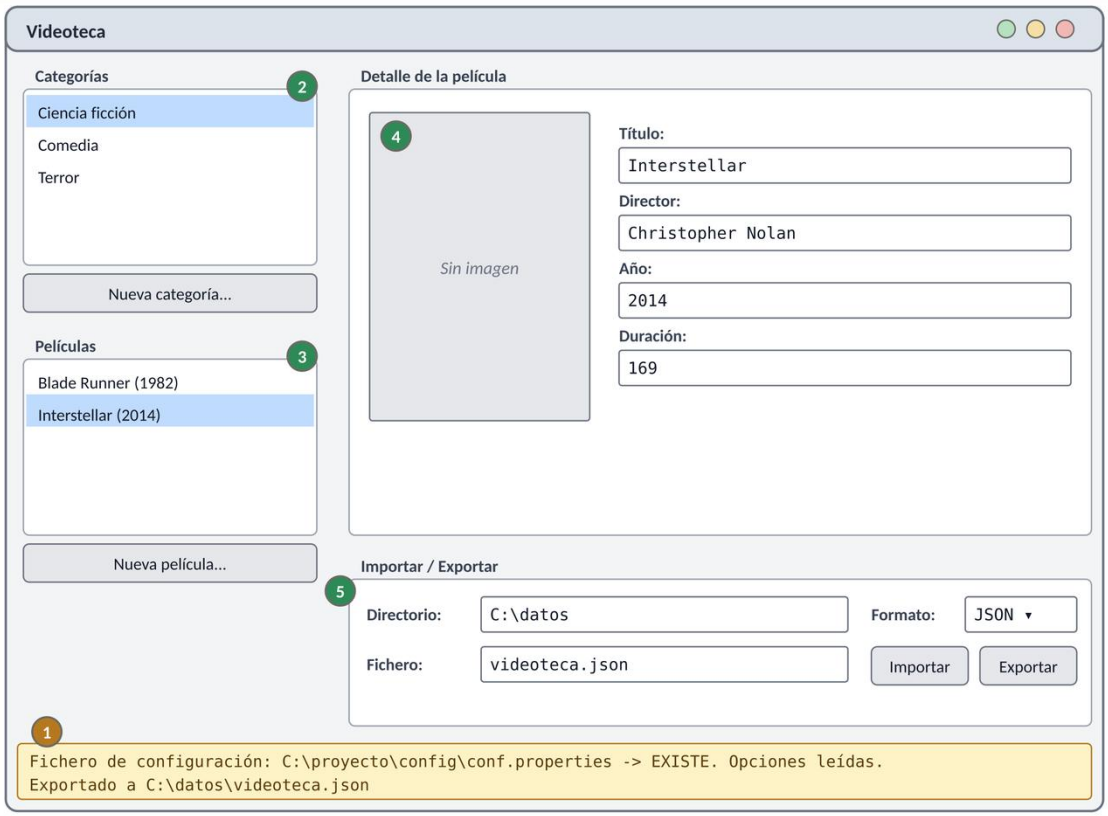
Cada capa **solo conoce a la capa de debajo**. Las **excepciones se propagan hacia arriba** hasta la UI, que enseña su mensaje.

Capa	Qué hace	¿Hecha?
1 · UI	Dibuja la ventana y recoge lo que hace el usuario	**Hecha**
2 · Contrato	La lista de operaciones que la UI puede pedir	**Hecha**
3 · Negocio	Datos en memoria y orden de llamadas	**Sin Hacer**
4 · Persistencia	Lee y escribe cada tipo de fichero	**Sin Hacer**
5 · Ficheros	Lo que queda en disco	—


EQUIPO A · YA HECHO
EQUIPO B · TÚ
CAPA 1 · UI (la ventana Swing)
DADA
Dibuja la pantalla y recoge lo que hace el usuario
Aplicacion
VentanaPrincipal
solo habla con la interfaz Java
CAPA 2 · CONTRATO (la interface Java)
DADA
La frontera entre los dos equipos: lista de operaciones
ServicioVideoteca
Formato
la implementas tú
CAPA 3 · NEGOCIO (las reglas)
EN PARTE TUYA
Datos en memoria y orden de llamadas (en parte ya hecho)
ServicioVideotecaImpl
GeneradorIds
CAPA 4 · PERSISTENCIA (leer y escribir ficheros)
TUYA
Cada clase sabe tratar un tipo de fichero
ComprobacionRutas
Configuracion
GestorTextoPlano
GestorJSON
GestorXML
AlmacenImagenes
CAPA 5 · FICHEROS
EN DISCO
Lo que queda en disco
conf.properties
imagenes.dat
indice.dat
.txt / .json / .xml
TRANSVERSAL
las usan TODAS las capas
MODELO
Categoria
Pelicula
Videoteca
EXCEPCIONES
ExcepcionDirectorio NoExiste
ExcepcionFichero NoExiste
ExcepcionOperacion Cancelada
Las excepciones
SUBEN por las capas
hasta la ventana

3 · CAPA 1 · LA VENTANA (no la tocas)

Los **números** dicen qué operación del contrato se ejecuta en cada sitio.

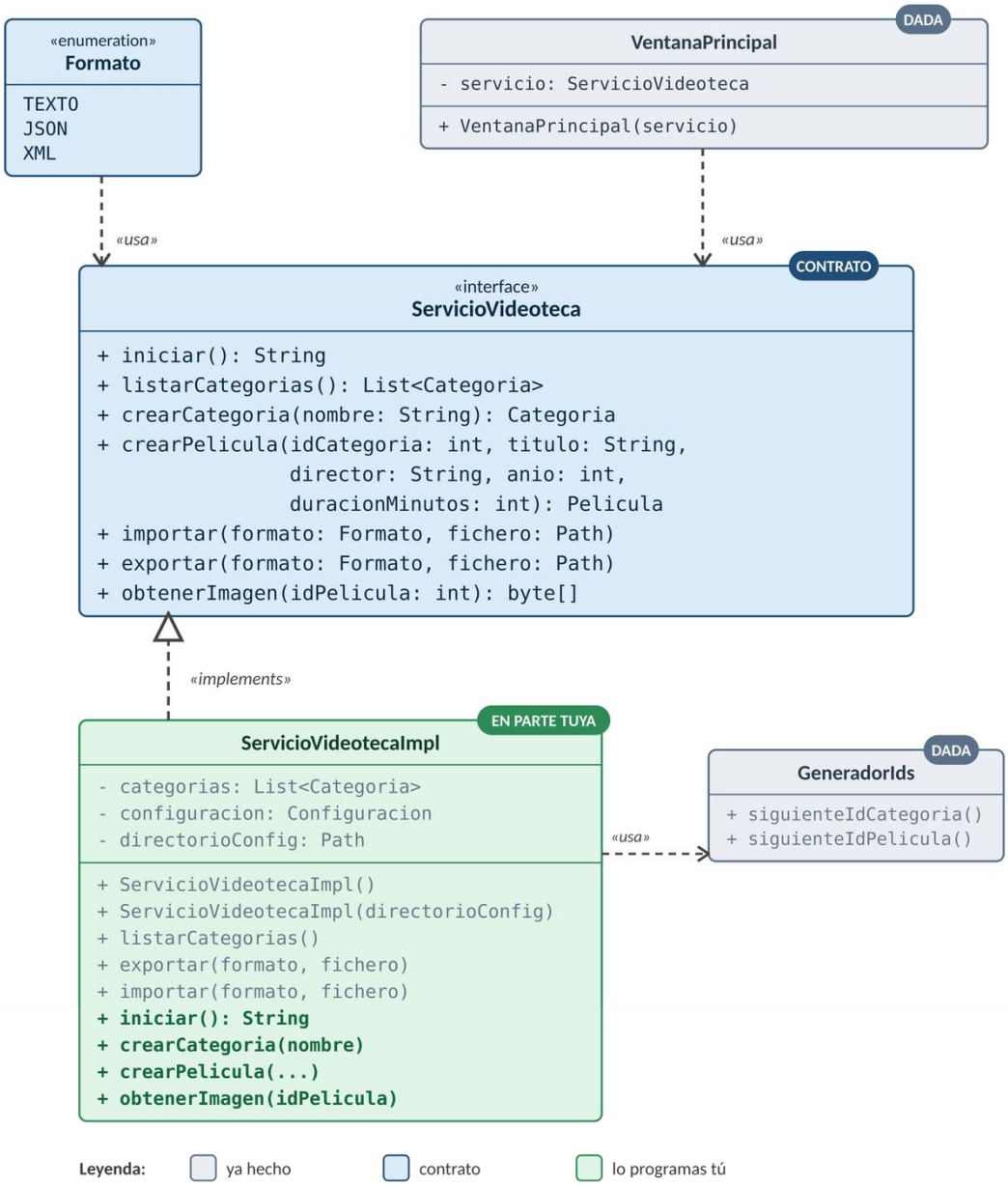


Nº	Qué haces en la ventana	Método del contrato
1	Se abre la ventana (barra de estado)	`iniciar()`
2	Ver categorías / «Nueva categoría»	`listarCategorias()` · `crearCategoria(...)`
3	Ver películas / «Nueva película»	`crearPelicula(...)`
4	Seleccionar una película (su carátula)	`obtenerImagen(id)`
5	Escribir directorio y fichero → Importar / Exportar	`importar(...)` · `exportar(...)`

- **`No hay JFileChooser`**: el directorio y el fichero se **escriben a mano**.
- **«Sobrescribir» no está en la ventana**: viene del `conf.properties`.
- Si algo **lanza una excepción**, la ventana enseña su **mensaje**: que sea **claro**.
- Si un método aún tiene su **TODO**, la ventana dice «**pendiente**» y **no se rompe**.

4 · CAPA 2 · EL CONTRATO: ServicioVideoteca

Es la **frontera** entre los dos equipos. La ventana **solo conoce esto**.



Método	Qué hace	Lanza
iniciar()	Una vez al arrancar: lee `./config/conf.properties` si existe y **devuelve un mensaje**	**Nada**
listarCategorias()	Categorías en memoria **con sus películas**	—
crearCategoria(nombre)	Categoría nueva (id automático)	`IllegalArgumentException`
crearPelicula(...)	Película nueva en **esa** categoría (id automático)	`IllegalArgumentException`

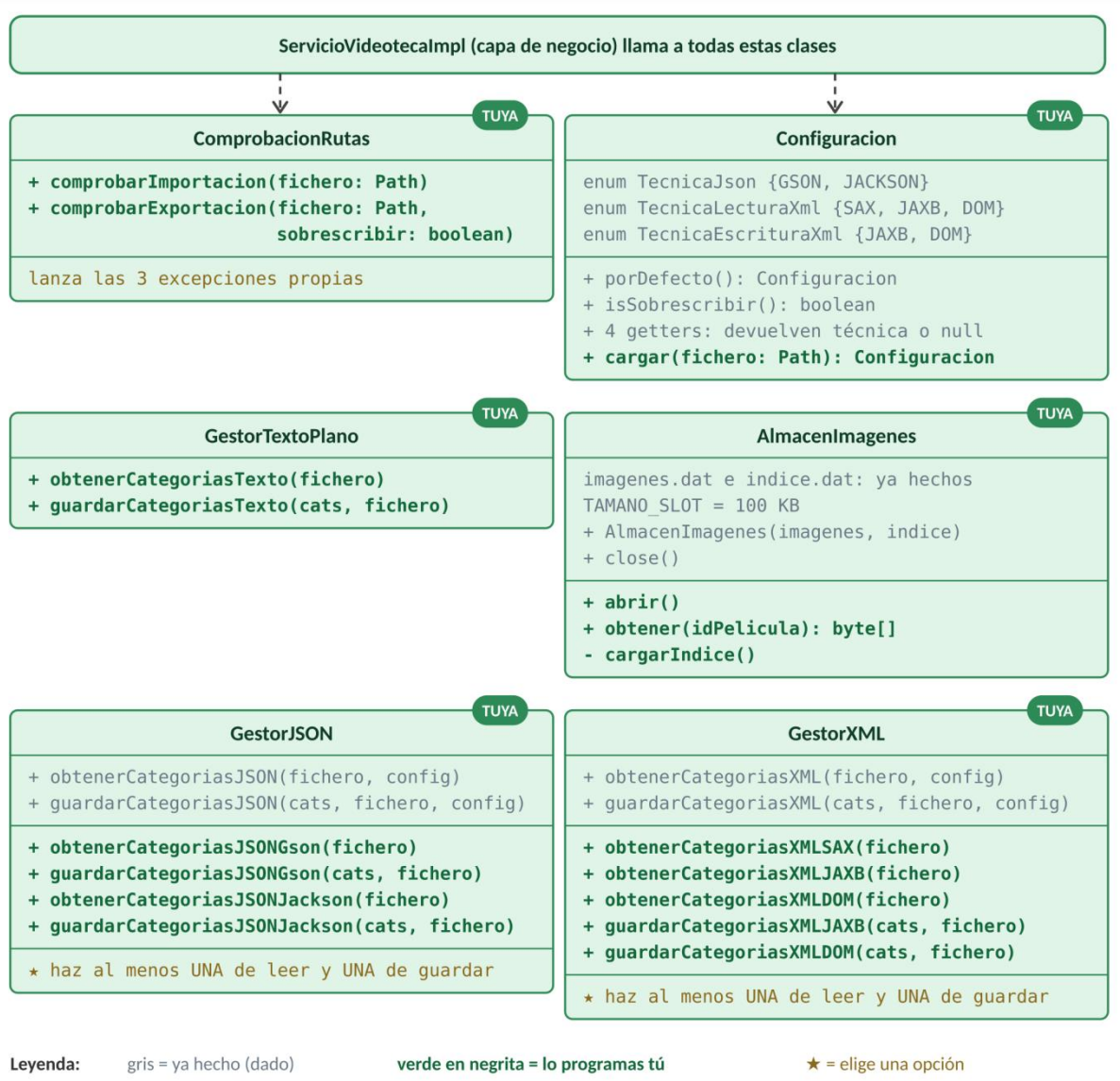
Método	Qué hace	Lanza
importar(formato, fichero)	Lee el fichero y **reemplaza** los datos en memoria	`IOException` · `ExcepcionDirectorioNoExiste` · `ExcepcionFicheroNoExiste`
exportar(formato, fichero)	Escribe los datos en memoria	`IOException` · `ExcepcionDirectorioNoExiste` · `ExcepcionOperacionCancelada`
obtenerImagen(id)	Devuelve la carátula, o **`null`**	`IOException`

- **Datos de ejemplo:** la videoteca arranca con **3 categorías y 4 películas** en memoria.
- `importar` **reemplaza** la memoria **solo si la lectura sale bien**. Si falla, no cambia nada.
- `exportar` **no pregunta** si sobrescribir: lo dice el `.properties` (**`por defecto false`**).
- Cualquier método puede lanzar también `UnsupportedOperationException` (está **pendiente de implementar**), `IllegalStateException` (**sin configurar**) o `IllegalArgumentException` (**datos no válidos**).

Ya hechas: las **tres excepciones propias** ya están creadas (`videoteca.excepciones`). **Solo hay que usarlas**.

5 · TUS CAPAS: QUÉ ESTÁ HECHO Y QUÉ PROGRAMAS

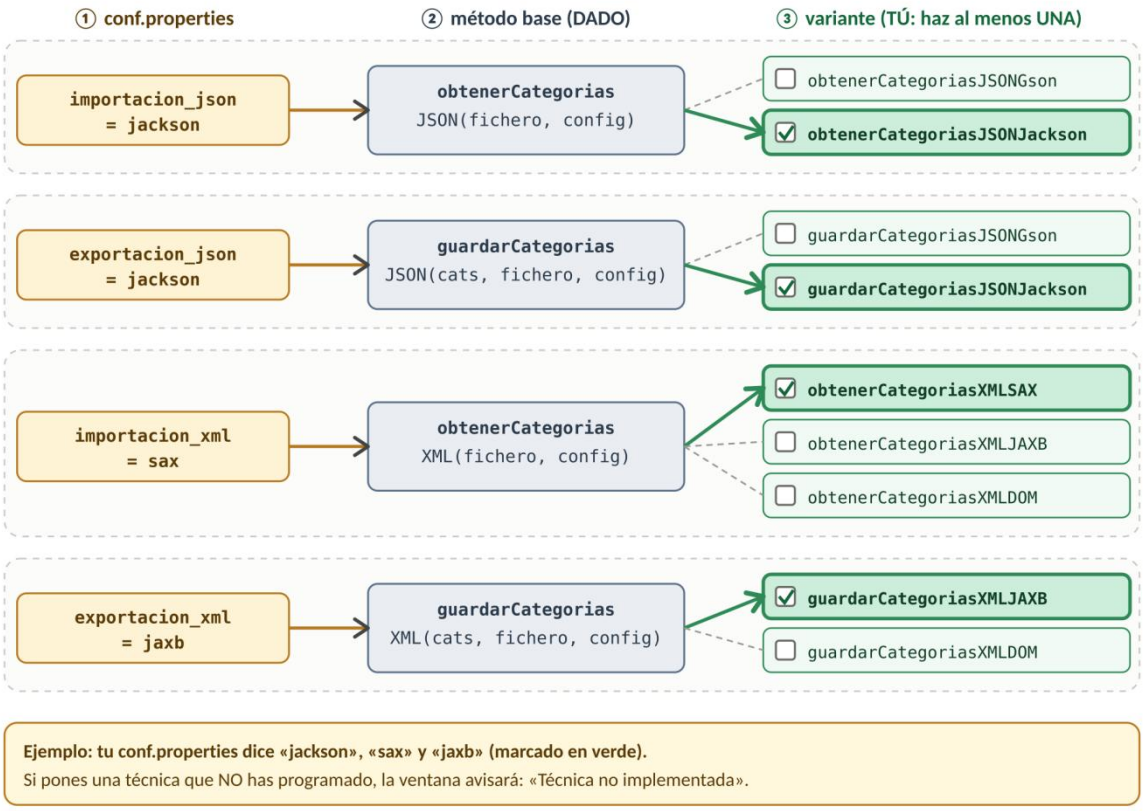
Gris = ya hecho. **Verde en negrita** = lo haces tú. ★ = elige solo una opción.



Clase	Ya hecho	Lo programas tú
ServicioVideotecaImpl	`listarCategorias`, **`exportar`**, **`importar`** y constructores	`iniciar`, `crearCategoria`, `crearPelicula`, `obtenerImagen`
GeneradorIds	**Todo** (id = mayor + 1 sobre **todas** las películas)	—
ComprobacionRutas	—	`comprobarExportacion`, `comprobarImportacion`
Configuracion	Enumerados, constructor, `porDefecto()` y getters	`cargar(fichero)`
GestorTextoPlano	—	`guardarCategoriasTexto`, `obtenerCategoriasTexto`
GestorJSON	Métodos **base** (eligen la variante)	Variantes **Gson** o **Jackson** (una)

Clase	Ya hecho	Lo programas tú
GestorXML	Métodos **base**	Lectura: **SAX**, **JAXB** o **DOM** (una). Escritura: **JAXB** o **DOM** (una)
AlmacenImagenes	Constructor, `close()`, `TAMANO_SLOT` y los ficheros `imagenes.dat` e `indice.dat`	`abrir`, `obtener` e índice (**solo lectura**)

Los métodos base miran tu `conf.properties` y llaman a la variante. **Tú solo rellenas la variante que elijas.**



6 · LAS 9 TAREAS

	Tarea	Qué verás en la ventana
☐	**1** · Exportar a texto plano	**Exportas** los datos de ejemplo
☐	**2** · Exportar a JSON	Exportas JSON
☐	**3** · Exportar a XML	Exportas XML
☐	**4** · Importar de texto plano	**Importas** y cambian las listas
☐	**5** · Importar de JSON	Importas JSON
☐	**6** · Importar de XML	Importas XML
☐	**7** · Configuración y arranque	El **mensaje** de la barra de estado
☐	**8** · Imágenes	Muestras las carátulas
☐	**9** · Crear categorías y películas	**Creas** datos nuevos

Ojo: hasta la tarea 7 la ventana no sabe **qué librería** usar para JSON y XML.

BLOQUE A · EXPORTAR

TAREA 1 · Exportar a texto plano

Qué programas	**ComprobacionRutas.comprobarExportacion** y **GestorTextoPlano.guardarCategoriasTexto**
Apuntes	**Apartado 2** (NIO.2), **apartado 4** (texto) y **apartado 14** (excepciones)

comprobarExportacion(fichero, sobrescribir)

1. El **directorio** del fichero tiene que existir → si no --> `ExcepcionDirectorioNoExiste(directorio)`.
2. Si el fichero **no existe** → no pasa nada, **se creará**.
3. Si **existe** y `sobrescribir` es **false** → `ExcepcionOperacionCancelada(fichero)`.
4. Si **existe** y `sobrescribir` es **true** → no pasa nada, **se sobrescribirá**.

guardarCategoriasTexto(categorias, fichero)

```
CATEGORIA;id;nombre
PELICULA;id;titulo;director;anio;duracionMinutos

CATEGORIA;1;Ciencia ficción
PELICULA;101;Blade Runner;Ridley Scott;1982;117
CATEGORIA;3;Terror
```

1. Una línea **CATEGORIA** por categoría y, **debajo**, una **PELICULA** por cada una de sus películas.

TAREA 2 · Exportar a JSON (elige UNA)

Qué programas	**GestorJSON**: una variante de **guardar**
Apuntes	**Apartado 12** (Gson y Jackson)

☐	Variante	Librería
☐	`guardarCategoriasJSONGson(cats, fichero)`	**Gson**
☐	`guardarCategoriasJSONJackson(cats, fichero)`	**Jackson**

TAREA 3 · Exportar a XML (elige UNA)

Qué programas	**GestorXML**: una variante de **guardar**
Apuntes	**Apartado 8** (DOM) y **apartado 10** (JAXB)

☐	Variante	Técnica
☐	`guardarCategoriasXMLJAXB(cats, fichero)`	**JAXB** (menos código)
☐	`guardarCategoriasXMLDOM(cats, fichero)`	**DOM** (más trabajo)

Formato (todo son atributos)

```
<videoteca>
    <categoria id="1" nombre="Ciencia ficción">
        <pelicula id="101" titulo="Blade Runner" director="Ridley Scott"
                  anio="1982" duracionMinutos="117"/>
    </categoria>
    <categoria id="3" nombre="Terror"/>
</videoteca>
```

BLOQUE B · IMPORTAR

TAREA 4 · Importar de texto plano

Qué programas	**ComprobacionRutas.comprobarImportacion** y **GestorTextoPlano.obtenerCategoriasTexto**
Apuntes	**Apartado 2**, **apartado 4** y **apartado 14**

comprobarImportacion(fichero)

1. Saca el directorio: `fichero.toAbsolutePath().getParent()`.
2. Si **no es un directorio** → `ExcepcionDirectorioNoExiste(directorio)`.
3. Si **no es un fichero** (`Files.isRegularFile`) → `ExcepcionFicheroNoExiste(fichero)`.

obtenerCategoriasTexto(fichero)

1. Lee **línea a línea**, ignora las líneas en blanco y separa por ";".
2. **CATEGORIA** → crea una categoría, añádela a la lista y recuérdala (**«es la actual»**).
3. **PELICULA** → créala y añádela a **la categoría actual**.
4. **Línea rara** (desconocida, PELICULA antes de una CATEGORIA, número mal escrito,etc) → **IOException** con mensaje claro.
5. Devuelve la lista.

Truco: exporta primero (tarea 1) y luego **importa ese mismo fichero**: los datos tienen que salir **iguales**.

TAREA 5 · Importar de JSON (elige UNA)

Qué programas	**GestorJSON**: una variante de **obtener**
Apuntes	**Apartado 12**

☐	Variante	Librería
☐	`obtenerCategoriasJSONGson(fichero)`	**Gson**
☐	`obtenerCategoriasJSONJackson(fichero)`	**Jackson**

Gson

Si el JSON está mal, Gson lanza `JsonParseException` (**`no es IOException`**): **captúrala** y lanza una `IOException` con mensaje claro.

Jackson

Jackson **`ya lanza IOException`** por sí mismo: no hay que convertir nada.

TAREA 6 · Importar de XML (elige UNA)

Qué programas	**GestorXML**: una variante de **obtener** (y `VideotecaHandler` si eliges SAX)
Apuntes	**Apartados 8, 9 y 10** (DOM, SAX, JAXB)

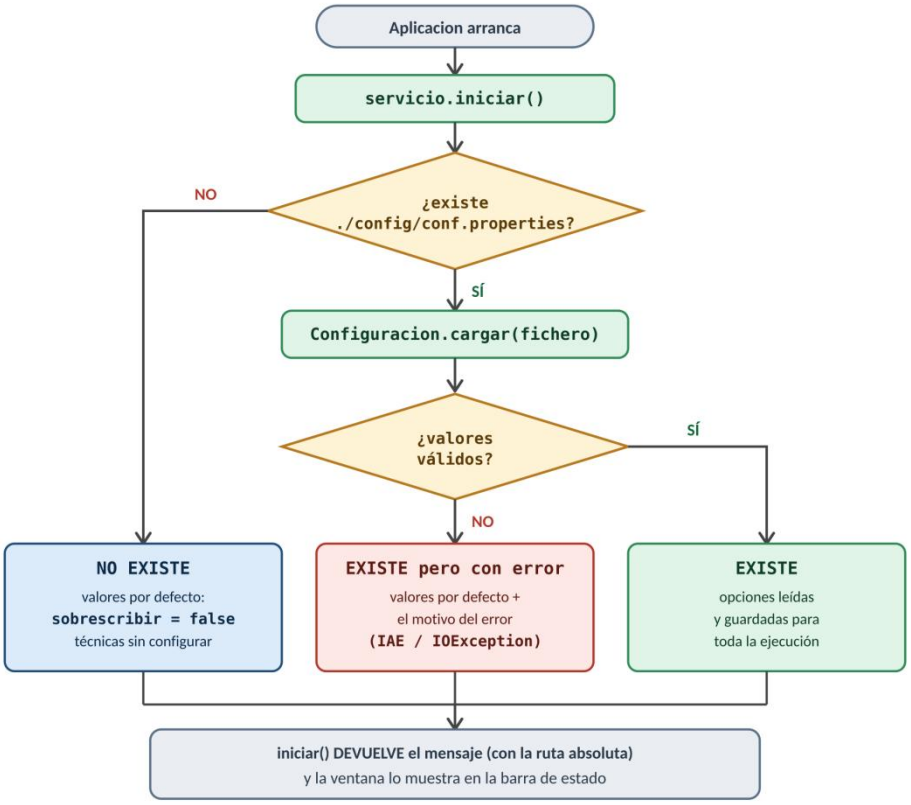
☐	Variante	Técnica
☐	`obtenerCategoriasXMLSAX(fichero)`	**SAX** (como UD1 EP 29)
☐	`obtenerCategoriasXMLJAXB(fichero)`	**JAXB** (menos código)
☐	`obtenerCategoriasXMLDOM(fichero)`	**DOM**

BLOQUE C · CONFIGURACIÓN E IMÁGENES

TAREA 7 · Configuración (.properties) y arranque

Qué programas	**Configuracion.cargar(fichero)** y **ServicioVideotecaImpl.iniciar()**
Apuntes	**Apartado 13** (ficheros .properties)
Terminada cuando	**T7.a** a **T7.f** en **[OK]**. **Ya funcionan JSON y XML en la ventana.**

La ruta es **fija e inmutable**: **./config/conf.properties**. Se lee **una sola vez**, al arrancar.



Clave	Valores	Si falta
sobrescribir	`true` o `false`	**false**
importacion_json	`gson` o `jackson`	**gson**
exportacion_json	`gson` o `jackson`	**gson**
importacion_xml	`sax`, `jaxb` o `dom`	**dom**
exportacion_xml	`jaxb` o `dom` (**SAX no puede escribir**)	**dom**

Configuracion.cargar(fichero)

Devuelve `new Configuracion(...)` con los valores obtenidos o los valores por defecto si no se obtiene nada.

iniciar()

1. Ruta: `directorioConfig.resolve("conf.properties")`. Construye el **mensaje** con la **ruta absoluta** (`toAbsolutePath().normalize()`).
2. **Existe** → Guárdala en `configuracion` y el mensaje contiene **EXISTE**.
3. **No existe** → `Configuración por defecto` y el mensaje contiene **NO EXISTE**.
4. **Existe pero falla al leerla** → valores por defecto; el mensaje contiene **EXISTE** y **el motivo**.
5. **iniciar() nunca lanza excepciones**: todo va **dentro del mensaje**.

Ejemplo de conf.properties

```
# true: sobrescribe al exportar | false (por defecto): cancela si ya existe
sobrescribir=false

# JSON: gson | jackson
importacion_json=jackson
exportacion_json=jackson

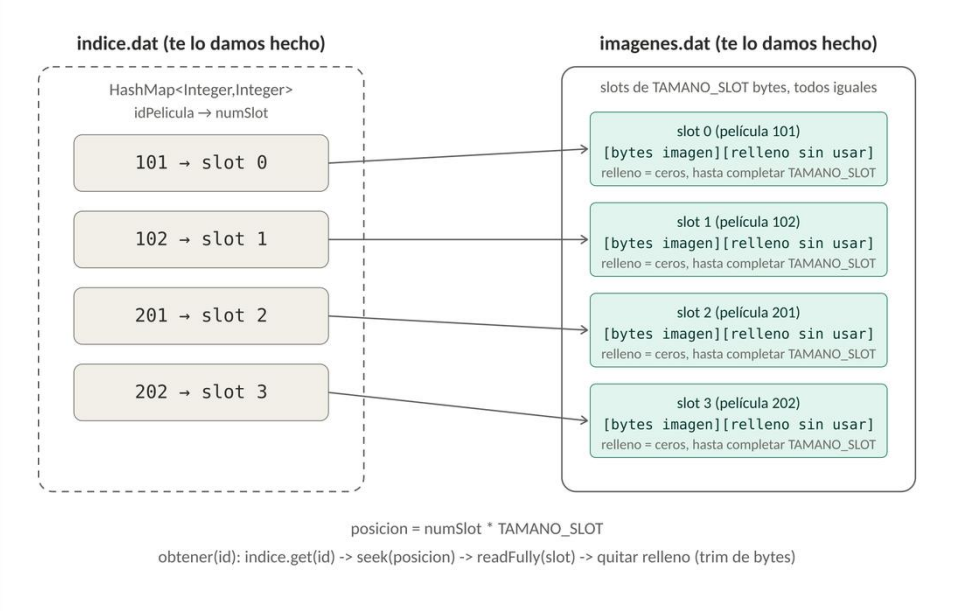
# XML lectura: sax | jaxb | dom      XML escritura: jaxb | dom
importacion_xml=sax
exportacion_xml=jaxb
```

Ojo: los **comentarios** (#) van en **líneas aparte**. Un # al final de un valor **forma parte del valor**.

TAREA 8 · Imágenes en un fichero de acceso directo

Qué programas	**AlmacenImagenes** y **obtenerImagen** del servicio
Apuntes	**Apartado 6** (RandomAccessFile), **apartado 7** (serialización), **UD1 EP 13** (longitud fija)

Los ficheros **imagenes.dat** e **indice.dat se te dan hechos**, con las carátulas de las 4 películas de ejemplo. Tú solo tienes que **buscar** la imagen de una película (no hay inserción/eliminación):



Pieza	Regla
imagenes.dat	**Registros** de `TAMANO_REGISTRO` bytes (100 KB), **todos iguales**
Un slot	**[bytes de la imagen][relleno de ceros]** — **sin longitud** por delante
Posición del registro n	**n × TAMANO_REGISTRO** (usa `(long)`)
indice.dat	`HashMap<Integer,Integer>` **serializado**: idPelicula → nº de registro
Al leer	Lee el **registro entero** y **quita los ceros del final**

AlmacenImagenes

1. **abrir()**: `RandomAccessFile` en modo `"r"` (**solo lectura**) y `indice = cargarIndice()`.
2. **cargarIndice()**: lee con *`ObjectInputStream`* el `HashMap<Integer,Integer>` de `indice.dat`.
3. **obtener(idPelicula)**: si el id **no está** en el índice → **`null`**.
4. Si está: posición = **n × TAMANO_SLOT** (usa `(long)`), `seek(posición)` y `readFully` de **un slot entero**.
5. **Quita los ceros del final** (recorre el array desde el final mientras el byte sea 0) y devuelve la parte útil con `Arrays.copyOf`.

¿Por qué se pueden quitar los ceros? quitar ceros finales solo es correcto si la imagen **nunca termina en 0**. Las carátulas guardadas son **PNG**, que acaban siempre en 82.

En el servicio

1. Las rutas son `directorioConfig/imagenes.dat` y `directorioConfig/indice.dat`. Deberán buscarse esos ficheros tanto en el propio directorio como en cualquier subdirectorio dentro de él de forma recursiva (`directorioConfig/.../imagenes.dat`, `directorioConfig/.../.../imagenes.dat`, `etc`).
2. **obtenerImagen(id)**: si **no existe** `imagenes.dat` o `indice.dat` → **`null`** (no es un error). Si existen, `obtener(id)`.
3. Abre y **cierra** el almacén **en cada operación** (`try-with-resources`).

BLOQUE D · DATOS EN MEMORIA

TAREA 9 · Crear categorías y películas

Qué programas	**crearCategoria** y **crearPelicula** en `ServicioVideotecaImpl`
Apuntes	Colecciones (`List`) y POO

Los `ids ya los calcula GeneradorIds` (mayor + 1 sobre **todas** las películas): **solo tienes que llamarlo**.

crearCategoria(nombre)

1. Si el nombre es `null` o está en blanco → **IllegalArgumentException**.
2. id = `GeneradorIds.siguienteIdCategoria(categorias)`.
3. `new Categoria(id, nombre)`, **añádela** a `categorias` y **devuélvela**.

crearPelicula(idCategoria, titulo, director, anio, duracionMinutos)

1. Si falta el título → **IllegalArgumentException**.
2. Busca la categoría por su **id**. Si **no existe** → **IllegalArgumentException**.
3. id = `GeneradorIds.siguienteIdPelicula(categorias)`.
4. Crea la `Pelicula`, **añádela a esa categoría** y **devuélvela**.

7 · FICHEROS Y EJECUCIÓN

```
<carpeta del proyecto>/
├── pom.xml
├── src/...
└── config/                ← la CREAS tú, a mano
    ├── conf.properties    ← lo escribes tú (tarea 7)
    ├── imagenes.dat       ← te lo damos hecho
    └── indice.dat         ← te lo damos hecho
```

- `./config/` es **relativa a la carpeta desde la que ejecutas** (en el IDE, la **raíz del proyecto**). Por eso el mensaje de arranque enseña la **ruta absoluta**.
- **Importar y exportar** usan **el directorio que escribas en la ventana**, pero **tiene que existir**: tu código **no crea carpetas**.

El `pom.xml` ya trae **todas** las librerías: abre el proyecto como **proyecto Maven**.

8 · ENTREGA Y EVALUACIÓN

Entregas el proyecto (`src` y `pom.xml`) en un **ZIP**, **sin** tu carpeta `config/`. **El código se corrige leyéndolo**: se valora **qué has implementado y cómo usas las librerías**.

Se valora	Cómo se ve
Uso de las librerías	Gson/Jackson y SAX/JAXB/DOM bien usadas; flujos cerrados con **try-with-resources**
Capas separadas	Los ficheros **solo** en persistencia; el servicio **no conoce** las librerías
Excepciones	Las **3 propias** se lanzan y **suben** hasta la ventana, con **mensajes claros**
Configuración	Se lee **una vez**; `sobrescribir` **false** por defecto; técnica sin configurar controlada
Almacén de imágenes	Lectura correcta del **registro** (posición y ceros finales) y del **índice** serializado

Ampliaciones (opcional)

- Programa **otra técnica** de JSON o XML y cambia el `conf.properties` para probarla.
- Añade **modificar** y **eliminar**. Pista: ¿qué pasa con los ids si borras la última película?

ANEXO · RECORDATORIO DE RECURSIVIDAD (imagenes.dat e indice.dat)

Un método recursivo es aquel que **se llama a sí mismo** para resolver una versión **más pequeña** del mismo problema.

Las 3 reglas

	Regla	Si falta…
☐	**Caso base:** una situación en la que el método **ya no se llama a sí mismo**	`StackOverflowError`
☐	**Paso recursivo:** el método se llama a sí mismo con un problema **más pequeño**	no hay recursividad
☐	**Avanzar:** cada llamada se **acerca** al caso base	bucle infinito

Ejemplo mínimo con números

```
static int sumar(int n) {
    if (n == 0) return 0;        // caso base
    return n + sumar(n - 1);     // paso recursivo (n - 1 es más pequeño)
}

// sumar(3) = 3 + sumar(2) = 3 + 2 + sumar(1) = 3 + 2 + 1 + sumar(0) = 6
```

Recursividad y directorios

Un directorio contiene **ficheros** y **otros directorios**. Cada subdirectorio es **el mismo problema, más pequeño**. La plantilla es siempre esta:

```
método(directorio):
    si el directorio no es válido          → caso base
    para cada elemento del directorio:
        si es un fichero                   → hacer algo con él
        si es un directorio                → método(elemento)   ← paso recursivo
```

Ejemplo resuelto (distinto al que se pide en la práctica): contar ficheros

Cuenta cuántos ficheros hay en un directorio **y en todos sus subdirectorios**.

```
static int contarFicheros(File directorio) {
    if (directorio == null || !directorio.isDirectory()) return 0;   // caso base
    File[] elementos = directorio.listFiles();
    if (elementos == null) return 0;   // caso base: no se puede leer

    int total = 0;
    for (File elemento : elementos) {
        if (elemento.isFile()) {
            total++;
        } else {
            total += contarFicheros(elemento);   // paso recursivo
        }
    }
    return total;
}
```

Errores típicos

- **Olvidar el caso base** → `StackOverflowError`.
- **Llamar al método recursivo y no usar lo que devuelve**: el resultado se pierde.

- **No devolver nada al terminar el bucle**: hace falta un valor por defecto.
- **`listFiles()` puede devolver `null`** (sin permisos o error de lectura): hay que comprobarlo!.
- `Files.walk` y `Files.find` hacen lo mismo, pero aquí se trata de **practicar la recursividad**: tienes que implementarlo tú.

Cómo probarlo

Crea esta estructura y comprueba que encuentra el fichero **en cada nivel**:

```
config/
├── conf.properties
└── antiguas/
    └── 2025/
        ├── imagenes.dat
        └── indice.dat
```